

WorldReady

"Bob built this app English-only. Bob made it world-ready."

English + Français (Québec) + العربية (right-to-left)

Live: chinmoypaul8897.github.io/worldready

IBM Bob 2.0 Hackathon · lablab.ai · September 2026

Going global breaks the code, not just the copy

- Entering a new market is a **code problem**, not a translation problem.
- Quebec's **Bill 96** (French) and Gulf/MENA growth (Arabic) demand real localization — Arabic means **right-to-left** layout, **6 plural forms**, Eastern formatting.
- Machine translation can't fix hard-coded strings, glued plurals ("1 seats left"), physical left/right layout, or English baked into data and toast messages.
- One hard-coded string shipped later **silently breaks** the new market.
- Baseline (measured): **305** hard-coded user-visible strings · **1** language · `<html lang="en">` with no dir · **12** physical left/right classes.

A Bob-driven i18n retrofit — with a gate

- Turns an English-only React app into **English + French (Québec) + Arabic (RTL)**.
- Extracts every user-visible string into **i18next keys** — parallel subagents, one per app area.
- Real i18n, not just words: Intl number/date/relative-time · **CLDR plurals** (Arabic's 6 forms) · logical RTL layout · mirrored icons · Noto Sans Arabic.
- A **translator round-trip spreadsheet** for native review.
- A **CI gate + commit hook** that fails the build on any new hard-coded string.

See it flip

- The live viewer: switch to العربية and the whole app **mirrors to RTL**.
- "Scan" outlines hard-coded English in both panes — **Before: many** • **After: 0**.
- chinmoypaul8897.github.io/worldready

The screenshot displays the WorldReady application interface, which is designed for managing localization projects. The top navigation bar includes language selection buttons (English, Français, العربية, Pseudo) and a 'Scan for hard-coded text' button. The main dashboard provides a summary of the project's status: 305 hard-coded strings, 24/30 traps fixed, 0 English content changes, 1 to 3 pseudo languages, and 16.89 Bobcoins used. Below this, a 'Scan (ar)' section shows the results of a scan for Arabic content, indicating 47 hard-coded strings before and 0 after, with a 'clean' status. The interface is split into two panes: 'BEFORE' (Before — IBM Galaxium (English only)) and 'AFTER' (After — WorldReady). The 'BEFORE' pane shows the original English version of the 'Galaxium Travels' app, with a 'Book a Flight' button and a 'Home' link. The 'AFTER' pane shows the localized Arabic version, with the text 'سافر إلى ما وراء النجوم' (Travel beyond the stars) and a 'الرئيسية' (Home) link. A table titled 'Arabic Plural Showcase' is displayed below the panes, showing the results of the localization process for various counts of flights. The table has columns for COUNT, EN, FR, AR, and AR CATEGORY. The 'AR CATEGORY' column shows the localized Arabic plural forms for different counts of flights, such as 'zero', 'one', 'two', 'few', 'many', and 'other'. The 'Evidence Drawer' is visible at the bottom of the interface.

COUNT	EN	FR	AR	AR CATEGORY
0	Showing 0 flights	0 vol affichés	لا توجد رحلات	zero
1	Showing 1 flight	1 vol affichés	رحلة واحدة (1)	one
2	Showing 2 flights	2 vols affichés	رحلتان (2)	two
3	Showing 3 flights	3 vols affichés	3 رحلات	few
11	Showing 11 flights	11 vols affichés	11 رحلة	many
100	Showing 100 flights	100 vols affichés	100 رحلة	other

Built end-to-end in IBM Bob 2.0

- **17** focused Bob tasks · **17.67** Bobcoins of 40 · every task screenshotted + exported in `bob_sessions/`.
- Plan mode · custom "**i18n-extractor**" mode (edit scope limited by fileRegex) · custom **i18n-extract** skill.
- **5** parallel subagents (one per app area) · document understanding (glossary.xlsx + style-guide.pdf) · Office-file write (translator sheet).
- **PreToolUse** commit-gate hook · **GitHub Actions** i18n gate.
- An honest **plain-Bob baseline** (no kit) is committed as a control.

Measured, not claimed

- **305** → **0** hard-coded user-visible English strings (independent verifier in `scripts/`).
- **24 / 30** engineered i18n traps fixed — scored against a HELD-OUT key whose SHA-256 was committed before any Bob task ran; no Bob prompt ever named a trap.
- **0** characters of existing English content changed (the language switcher is +45 chars of new UI).
- **key-parity PASS** across en/fr/ar/pseudo · 1 → 3 languages + a pseudo-locale.
- The gate is live: a public PR goes **RED** on a hard-coded string, **GREEN** after the i18next fix.

Who pays, and why us

- **Buyers:** product teams entering regulated/RTL markets (Quebec Bill 96, Gulf/MENA); agencies retrofitting client apps.
- **Alternatives** — Lingo.dev, i18next-parser, Crowdin — help extract or translate; none retrofit the code traps (plurals, formatting, RTL, data & toasts) **AND** install a regression gate in one pass.
- **Differentiation:** TransCreate localizes developer videos; WorldReady localizes app **code** — keys, plurals, formatting, RTL, and a gate. Accessibility tools such as Usher and CurbCut ask whether *everyone* can use the app; WorldReady asks whether *every market* can.

Honest limits

- Translations are **machine drafts** for native review — a translator round-trip sheet ships for exactly that.
- **6 of 30** traps remain, on error/format code paths off the happy path — reported honestly, never answer-fed.
- An early commit briefly contained the trap list (~3 minutes) before any Bob task ran; no Bob prompt referenced it; the plain-Bob baseline on the same folder is the fairness control.
- **Next:** more locales · RTL visual-regression in CI · a hosted "paste your repo" scan.

Thank you.

Live: chinmoypaul8897.github.io/worldready

Repo: github.com/chinmoypaul8897/worldready

Built with IBM Bob 2.0.